

Maximizing Revenue for Taxi Cab Drivers through Payment Type Analysis

Problem Statement

In the fast-paced taxi booking sector, making the most of revenue is essential for long-term success and driver happiness. Our goal is to use data-driven insights to maximise revenue streams for taxi drivers in order to meet this need. Our research aims to determine whether payment methods have an impact on fare pricing by focusing on the relationship between payment type and fare amount.

Objective

This project's main goal is to run an A/B test to examine the relationship between the total fare and the method of payment. We use Python hypothesis testing and descriptive statistics to extract useful information that can help taxi drivers generate more cash. In particular, we want to find out if there is a big difference in the fares for those who pay with credit cards versus those who pay with cash.

Research Question

Is there a relationship between total fare amount and payment type and can we nudge customers towards payment methods that generate higher revenue for drivers, without negatively impacting customer experience?

Importing Libraries

Phase 1: The "Dirty Data" Audit (Data Cleaning)

This is your first test. The engineering team warns that the sensors might be glitching. The Client asks:

1. The "Ghost" Rides: "We are seeing trips with `trip_distance = 0` but the meter was still running. How many of these exist? Are they valid (e.g., stuck in traffic) or system errors? Please analyse and handle them."
2. The Negative Money Problem: "Our accounting team found rows with negative `fare_amount` and `total_amount`. We can't have negative revenue. Identify these rows. Is it a specific `payment_type` causing this? Remove or fix them."
3. The Silent Passenger: "We have trips with `passenger_count = 0`. Is this possible? (maybe package delivery?). If it's less than 1% of data, remove it. If it's more, keep it and explain why."

4. The Time Travelers: "Check for trips where the dropoff_datetime is before the pickup_datetime. These are obvious errors."

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')
```

Loading the data

```
df = pd.read_csv(r"C:\Users\Admin\Documents\Deepak Documents\PROJECTS\Hypothesis Project\yellow_tripdata_2020-01.csv")
```

```
# taxi_data.loc[:900000].to_csv(r"C:\Users\RajendraPrasad\Downloads\Taxi_data.csv")
```

```
df.shape
```

```
(6405008, 18)
```

```
df.head()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime
passenger_count \			
0	1.0	2020-01-01 00:28:15	2020-01-01 00:33:03
1.0			
1	1.0	2020-01-01 00:35:39	2020-01-01 00:43:04
1.0			
2	1.0	2020-01-01 00:47:41	2020-01-01 00:53:52
1.0			
3	1.0	2020-01-01 00:55:23	2020-01-01 01:00:14
1.0			
4	2.0	2020-01-01 00:01:58	2020-01-01 00:04:16
1.0			

	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID
DOLocationID \				
0	1.2	1.0	N	238
239				
1	1.2	1.0	N	239
238				
2	0.6	1.0	N	238
238				
3	0.8	1.0	N	238
151				
4	0.0	1.0	N	193
193				

	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount
0	1.0	6.0	3.0	0.5	1.47	0.0
1	1.0	7.0	3.0	0.5	1.50	0.0
2	1.0	6.0	3.0	0.5	1.00	0.0
3	1.0	5.5	0.5	0.5	1.36	0.0
4	2.0	3.5	0.5	0.5	0.00	0.0

	improvement_surcharge	total_amount	congestion_surcharge
0	0.3	11.27	2.5
1	0.3	12.30	2.5
2	0.3	10.80	2.5
3	0.3	8.16	0.0
4	0.3	4.80	0.0

df.tail()

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
6405003	NaN	2020-01-31 22:51:00	2020-01-31 23:22:00	NaN
6405004	NaN	2020-01-31 22:10:00	2020-01-31 23:26:00	NaN
6405005	NaN	2020-01-31 22:50:07	2020-01-31 23:17:57	NaN
6405006	NaN	2020-01-31 22:25:53	2020-01-31 22:48:32	NaN
6405007	NaN	2020-01-31 22:44:00	2020-01-31 23:06:00	NaN

	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID
6405003	3.24	NaN	NaN	237
6405004	22.13	NaN	NaN	259
6405005	10.51	NaN	NaN	137
6405006	5.49	NaN	NaN	50
6405007	11.60	NaN	NaN	179

	DOLocationID	payment_type	fare_amount	extra	mta_tax	tip_amount
6405003	234	NaN	17.59	2.75	0.5	0.0
6405004	45	NaN	46.67	2.75	0.5	0.0
6405005	169	NaN	48.85	2.75	0.0	0.0

6405006	42	NaN	27.17	2.75	0.0
0.0					
6405007	205	NaN	54.56	2.75	0.5
0.0					

	tolls_amount	improvement_surcharge	total_amount	\
6405003	0.00	0.3	21.14	
6405004	12.24	0.3	62.46	
6405005	0.00	0.3	51.90	
6405006	0.00	0.3	30.22	
6405007	0.00	0.3	58.11	

	congestion_surcharge
6405003	0.0
6405004	0.0
6405005	0.0
6405006	0.0
6405007	0.0

Explortatory Data Analysis

Dataset Description

Field Name	Description
VendorID	A code indicating the TPEP provider that provided the record. 1= Creative Mobile Technologies, LLC; 2= VeriFone Inc.
tpep_pickup_datetime	The date and time when the meter was engaged.
tpep_dropoff_datetime	The date and time when the meter was disengaged.
Passenger_count	The number of passengers in the vehicle. This is a driver-entered value.
Trip_distance	The elapsed trip distance in miles reported by the taximeter.
PULocationID	TLC Taxi Zone in which the taximeter was engaged
DOLocationID	TLC Taxi Zone in which the taximeter was disengaged
RateCodeID	The final rate code in effect at the end of the trip. 1= Standard rate 2=JFK 3=Newark 4=Nassau or Westchester 5=Negotiated fare 6=Group ride
Store_and_fwd_flag	This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka “store and forward,” because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip
Payment_type	A numeric code signifying how the passenger paid for the trip. 1= Credit card 2= Cash 3= No charge 4= Dispute 5= Unknown 6= Voided trip
Fare_amount	The time-and-distance fare calculated by the meter.
Extra	Miscellaneous extras and surcharges. Currently, this only includes the \$0.50 and \$1 rush hour and overnight charges.
MTA_tax	\$0.50 MTA tax that is automatically triggered based on the metered rate in use.
Improvement_surcharge	\$0.30 improvement surcharge assessed trips at the flag drop. The improvement surcharge began being levied in 2015.
Tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
Tolls_amount	Total amount of all tolls paid in trip.
Total_amount	The total amount charged to passengers. Does not include cash tips.
Congestion_Surcharge	Total amount collected in trip for NYS congestion surcharge.
Airport_fee	\$1.25 for pick up only at LaGuardia and John F. Kennedy Airports

```
# rows and columns of the dataset
```

```
df.shape
```

```
(6405008, 18)
```

```
df.dtypes
```

```
VendorID          float64
tpep_pickup_datetime  object
tpep_dropoff_datetime  object
passenger_count    float64
trip_distance      float64
RatecodeID        float64
store_and_fwd_flag  object
PULocationID       int64
DOLocationID       int64
payment_type       float64
fare_amount        float64
extra              float64
mta_tax            float64
tip_amount         float64
tolls_amount       float64
improvement_surcharge float64
total_amount       float64
congestion_surcharge float64
dtype: object
```

```
# converting pickup and dropoff to datetime
```

```
df['tpep_pickup_datetime'] =  
pd.to_datetime(df['tpep_pickup_datetime'])  
df['tpep_dropoff_datetime'] =  
pd.to_datetime(df['tpep_dropoff_datetime'])
```

```
df.dtypes
```

```
# taxi_data
```

```
VendorID          float64
tpep_pickup_datetime  datetime64[ns]
tpep_dropoff_datetime  datetime64[ns]
passenger_count    float64
trip_distance      float64
RatecodeID        float64
store_and_fwd_flag  object
PULocationID       int64
DOLocationID       int64
payment_type       float64
fare_amount        float64
extra              float64
mta_tax            float64
tip_amount         float64
tolls_amount       float64
```

```
improvement_surcharge      float64
total_amount                float64
congestion_surcharge        float64
dtype: object
```

```
# Calculating duration from the pickup and dropoff datetime in minutes
```

```
# subtracting the pickup time from dropoff time to get duration
```

```
df['duration'] = df['tpep_dropoff_datetime'] -
df['tpep_pickup_datetime']
```

```
df.head()
```

```
VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count \
0      1.0 2020-01-01 00:28:15 2020-01-01 00:33:03
1.0
1      1.0 2020-01-01 00:35:39 2020-01-01 00:43:04
1.0
2      1.0 2020-01-01 00:47:41 2020-01-01 00:53:52
1.0
3      1.0 2020-01-01 00:55:23 2020-01-01 01:00:14
1.0
4      2.0 2020-01-01 00:01:58 2020-01-01 00:04:16
1.0
```

```
trip_distance RatecodeID store_and_fwd_flag PULocationID
DOLocationID \
0      1.2      1.0      N      238
239
1      1.2      1.0      N      239
238
2      0.6      1.0      N      238
238
3      0.8      1.0      N      238
151
4      0.0      1.0      N      193
193
```

```
payment_type fare_amount extra mta_tax tip_amount tolls_amount
\
0      1.0      6.0      3.0      0.5      1.47      0.0
1      1.0      7.0      3.0      0.5      1.50      0.0
2      1.0      6.0      3.0      0.5      1.00      0.0
3      1.0      5.5      0.5      0.5      1.36      0.0
4      2.0      3.5      0.5      0.5      0.00      0.0
```


	improvement_surcharge	total_amount	congestion_surcharge
duration			
0	0.3	11.27	2.5 0 days
00:04:48			
1	0.3	12.30	2.5 0 days
00:07:25			
2	0.3	10.80	2.5 0 days
00:06:11			
3	0.3	8.16	0.0 0 days
00:04:51			
4	0.3	4.80	0.0 0 days
00:02:18			

converting into minutes

```
df['duration'] = df['duration'].dt.total_seconds()/60
```

```
df.head()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime
passenger_count \			
0	1.0	2020-01-01 00:28:15	2020-01-01 00:33:03
1.0			
1	1.0	2020-01-01 00:35:39	2020-01-01 00:43:04
1.0			
2	1.0	2020-01-01 00:47:41	2020-01-01 00:53:52
1.0			
3	1.0	2020-01-01 00:55:23	2020-01-01 01:00:14
1.0			
4	2.0	2020-01-01 00:01:58	2020-01-01 00:04:16
1.0			

	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID
DOLocationID \				
0	1.2	1.0	N	238
239				
1	1.2	1.0	N	239
238				
2	0.6	1.0	N	238
238				
3	0.8	1.0	N	238
151				
4	0.0	1.0	N	193
193				

	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount
\						
0	1.0	6.0	3.0	0.5	1.47	0.0
1	1.0	7.0	3.0	0.5	1.50	0.0

2	1.0	6.0	3.0	0.5	1.00	0.0
3	1.0	5.5	0.5	0.5	1.36	0.0
4	2.0	3.5	0.5	0.5	0.00	0.0

	improvement_surcharge	total_amount	congestion_surcharge	duration
0	0.3	11.27	2.5	4.800000
1	0.3	12.30	2.5	7.416667
2	0.3	10.80	2.5	6.183333
3	0.3	8.16	0.0	4.850000
4	0.3	4.80	0.0	2.300000

datatypes of the data

df.dtypes

```
VendorID                float64
tpep_pickup_datetime    datetime64[ns]
tpep_dropoff_datetime   datetime64[ns]
passenger_count          float64
trip_distance            float64
RatecodeID              float64
store_and_fwd_flag       object
PULocationID             int64
DOLocationID             int64
payment_type             float64
fare_amount              float64
extra                    float64
mta_tax                  float64
tip_amount               float64
tolls_amount             float64
improvement_surcharge    float64
total_amount             float64
congestion_surcharge     float64
duration                 float64
dtype: object
```

There are so many columns in the dataset, but as per our problem statement, we only require some fields from the original data. Rest columns are nothing but the unwanted columns for this study. So we will simply remove those columns.

As the problem statement is revolving around the payment type, fare amount, and any other factor influencing the fare amount, we will filter the data to have only passenger count, trip distance, payment type, fare amount and duration of the trip.

```
# removing unwanted columns
df.drop(['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
        'RatecodeID', 'store_and_fwd_flag', 'PULocationID', 'DOLocationID', 'extra',
        'mta_tax', 'tip_amount', 'tolls_amount',
        'improvement_surcharge',
        'total_amount', 'congestion_surcharge'],
        axis = 1, inplace = True)

# filtered data with relevant columns essential for the analysis
df.head()
```

	passenger_count	trip_distance	payment_type	fare_amount	duration
0	1.0	1.2	1.0	6.0	4.800000
1	1.0	1.2	1.0	7.0	7.416667
2	1.0	0.6	1.0	6.0	6.183333
3	1.0	0.8	1.0	5.5	4.850000
4	1.0	0.0	2.0	3.5	2.300000

```
# check for missing values
df.isnull().sum()

passenger_count    65441
trip_distance        0
payment_type       65441
fare_amount         0
duration            0
dtype: int64

df.shape[0]

6405008

65441/df.shape[0]*100

1.021716132126611

# percentage of missing data to the total record of the data
print('Missing data %', round(65441/df.shape[0]*100,2))

Missing data % 1.02

# as the data has only 1% of data missing, we can simply drop the
records with missing values.
df.dropna(inplace = True)
```

```

# check for missing values
df.isnull().sum()

passenger_count    0
trip_distance      0
payment_type       0
fare_amount        0
duration           0
dtype: int64

df.dtypes

passenger_count    float64
trip_distance      float64
payment_type       float64
fare_amount        float64
duration           float64
dtype: object

# changing the passenger count and payment type data type to integer
# as they are present in float
df['passenger_count'] = df['passenger_count'].astype('int64')
df['payment_type'] = df['payment_type'].astype('int64')

df.dtypes

passenger_count    int64
trip_distance      float64
payment_type       int64
fare_amount        float64
duration           float64
dtype: object

df.duplicated()

0      False
1      False
2      False
3      False
4      False
...
6339562    True
6339563    False
6339564    False
6339565    True
6339566    False
Length: 6339567, dtype: bool

# check for the duplicate rows
df[df.duplicated()]

```

	passenger_count	trip_distance	payment_type	fare_amount
duration				
2056	1	0.00	2	7.0
0.000000				
2441	1	0.00	1	52.0
0.200000				
2446	2	1.70	1	9.5
13.066667				
2465	1	0.40	1	4.0
3.083333				
3344	1	1.20	1	6.0
5.350000				
...	...	...	...	...
...				
6339558	1	1.63	2	8.0
8.800000				
6339559	1	1.81	1	8.5
8.016667				
6339560	1	0.98	2	6.5
6.900000				
6339562	1	2.10	1	11.0
14.233333				
6339565	1	1.61	2	8.5
9.633333				

[3331706 rows x 5 columns]

removing duplicate rows as they will not contribute in analysis
df.drop_duplicates(inplace = True)

after removing missing values and duplicate rows, now we have this much records left
df.shape

(3007861, 5)

passenger count distribution
df['passenger_count'].value_counts(normalize = True)*100

passenger_count	
1	58.198102
2	19.035022
3	6.636011
5	6.293675
6	3.927176
4	3.604621
0	2.303298
7	0.000931
9	0.000598

```

8      0.000565
Name: proportion, dtype: float64

# payment type distribution
df['payment_type'].value_counts()

print(df['payment_type'].value_counts(normalize = True) * 100)

payment_type
1      67.826705
2      30.757306
3       0.872148
4       0.543808
5       0.000033
Name: proportion, dtype: float64

```

We will focus solely on payment types 'card' and 'cash,' denoted by 1 and 2 in the dataset. To ensure our analysis centers on these payment methods, we'll filter the data accordingly, excluding all other types.

Moreover, examining the distribution of passenger counts reveals that trips with more than 5 passengers are rare. Additionally, trips with 0 passengers are impossible, as we expect at least one passenger to pay the fare amount. Therefore, we will filter the passenger count to include only values ranging from 1 to 5.

```

# filtering for payment type 1 and 2
df = df[df['payment_type']<3]

# filtering for passenger count from 1 to 2
df = df[(df['passenger_count']>0)&(df['passenger_count']<6)]

df.shape

(2780283, 5)

# taxi_data.passenger_count.value_counts()

# replacing the payment type encoded value 1 and 2 to Card and Cash
df['payment_type'].replace([1,2],['Card','Cash'], inplace = True)

df.head()

   passenger_count  trip_distance payment_type  fare_amount  duration
0                1             1.2         Card           6.0    4.800000
1                1             1.2         Card           7.0    7.416667
2                1             0.6         Card           6.0    6.183333
3                1             0.8         Card           5.5    4.850000
4                1             0.0         Cash           3.5    2.300000

# descriptive statistics for data
df.describe()

```

	passenger_count	trip_distance	fare_amount	duration
count	2.780283e+06	2.780283e+06	2.780283e+06	2.780283e+06
mean	1.733386e+00	4.536729e+00	1.780567e+01	2.415478e+01
std	1.176652e+00	4.895890e+00	1.506997e+01	9.260031e+01
min	1.000000e+00	-2.218000e+01	-5.000000e+02	-2.770367e+03
25%	1.000000e+00	1.500000e+00	9.000000e+00	9.883333e+00
50%	1.000000e+00	2.730000e+00	1.300000e+01	1.573333e+01
75%	2.000000e+00	5.470000e+00	2.100000e+01	2.336667e+01
max	5.000000e+00	2.628800e+02	4.265000e+03	8.525117e+03

Upon reviewing the provided statistics, it's evident that the minimum values for trip distance, fare amount, and duration are negative, which is unrealistic and invalid for further analysis. Consequently, we will eliminate these negative values from the dataset.

Furthermore, observing the maximum and 50th percentile values, it's possible that the data contains significant outliers, particularly high values. These outliers need to be addressed and removed to ensure the integrity of the analysis.

```
# filtering the records for only positive values
```

```
df = df[df['fare_amount']>0]
df = df[df['trip_distance']>0]
df = df[df['duration']>0]
```

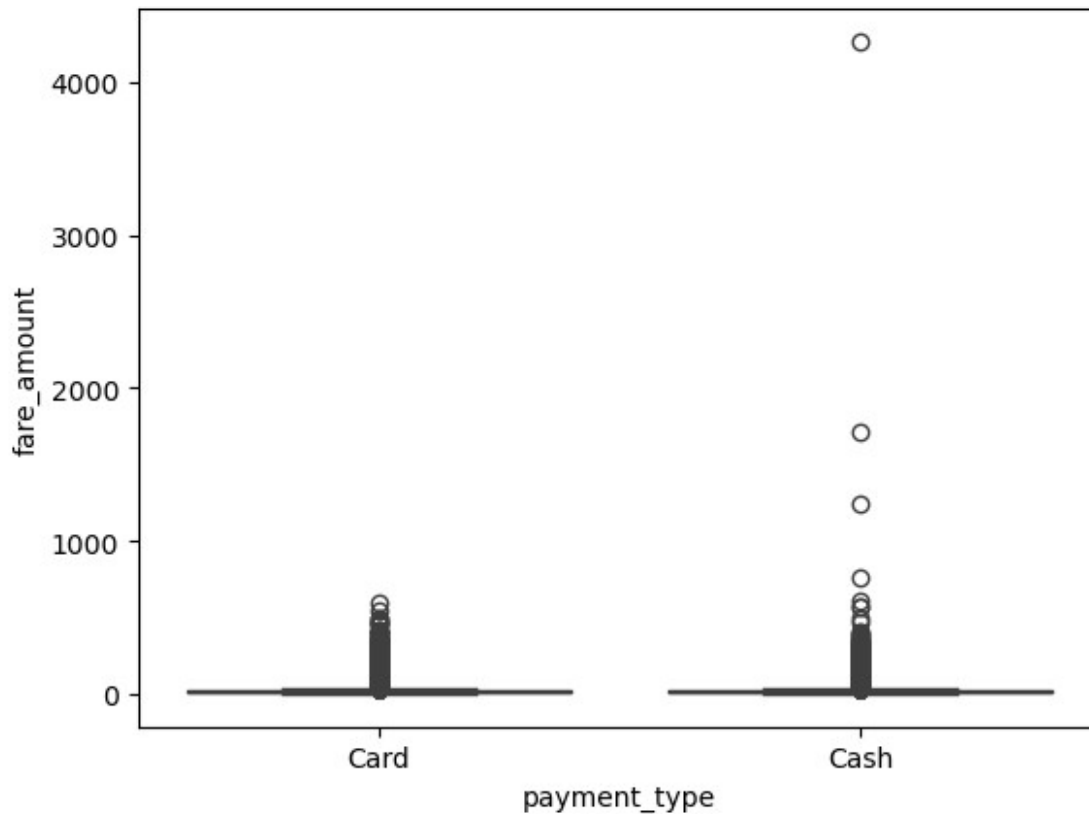
```
# descriptive statistics for data
```

```
df.describe()
```

	passenger_count	trip_distance	fare_amount	duration
count	2.748932e+06	2.748932e+06	2.748932e+06	2.748932e+06
mean	1.738151e+00	4.583313e+00	1.766835e+01	2.418261e+01
std	1.179440e+00	4.897806e+00	1.447531e+01	9.272285e+01
min	1.000000e+00	1.000000e-02	1.000000e-02	1.666667e-02
25%	1.000000e+00	1.530000e+00	9.000000e+00	9.950000e+00
50%	1.000000e+00	2.770000e+00	1.300000e+01	1.575000e+01
75%	2.000000e+00	5.500000e+00	2.100000e+01	2.333333e+01
max	5.000000e+00	2.628800e+02	4.265000e+03	8.525117e+03

```
# check for the outliers
```

```
sns.boxplot(data=df, y="fare_amount", x="payment_type")
plt.show()
```

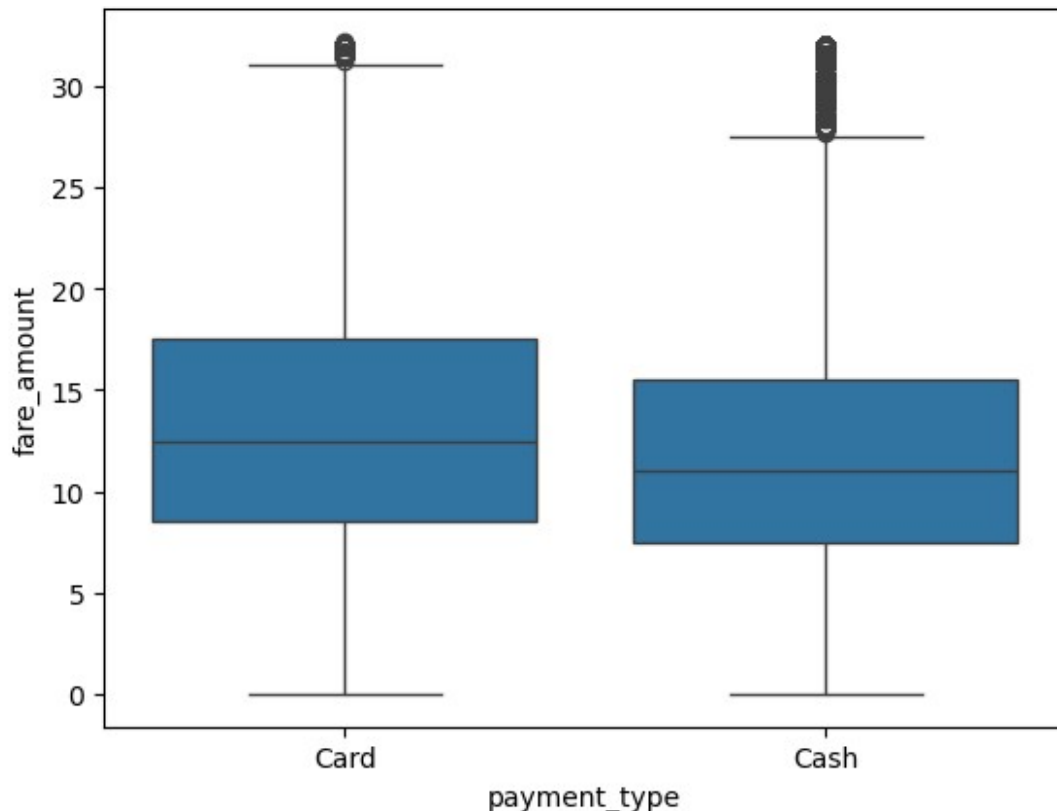


```
# Removing outliers using Interquartile Range (IQR) for numerical variables
for col in ['trip_distance', 'fare_amount', 'duration']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    # Define lower and upper bounds for outliers
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Filter out outliers
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

# check for the outliers
sns.boxplot(data=df, y="fare_amount", x="payment_type")
plt.show()
```

We're interested on exploring the relationship between payment type and passenger behavior concerning trip distance and fare amount. Are there variations in the distribution of payment types concerning different fare amounts or trip distances?

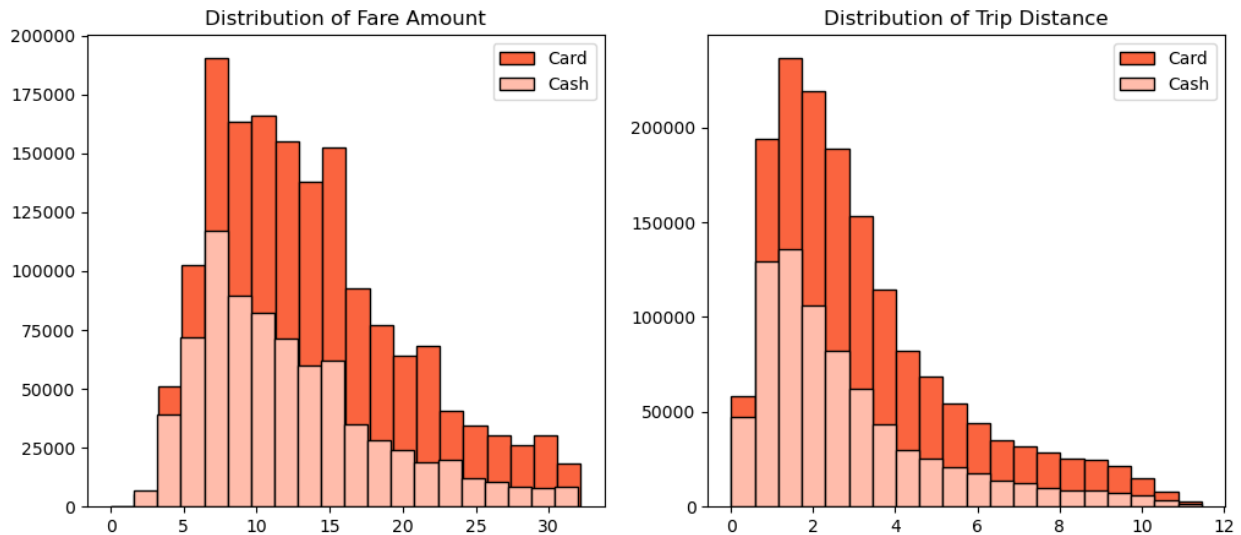
To investigate this, we'll plot histograms to visualize the distribution of passenger counts paying with either card or cash. This will also provide stakeholders with insight into fare amount ranges associated with different payment methods.

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.title('Distribution of Fare Amount')
plt.hist(df[df['payment_type']=='Card']['fare_amount'],
histtype='barstacked', bins = 20, edgecolor = 'k', color = '#FA643F',
label = 'Card')
plt.hist(df[df['payment_type']=='Cash']['fare_amount'],
histtype='barstacked', bins = 20, edgecolor = 'k', color =
'#FFBCAB', label = 'Cash')
plt.legend()

plt.subplot(1,2,2)
plt.title('Distribution of Trip Distance')
plt.hist(df[df['payment_type']=='Card']['trip_distance'],
histtype='barstacked', bins = 20, edgecolor = 'k', color =
'#FA643F', label = 'Card')
plt.hist(df[df['payment_type']=='Cash']['trip_distance'],
```

```
histtype='barstacked',bins = 20, edgecolor = 'k', color =
'#FFBCAB',label = 'Cash')
plt.legend()
plt.show()

# calculating the mean and standard deviation group by on payment type
df.groupby('payment_type').agg({'fare_amount': ['mean', 'std'],
'trip_distance': ['mean', 'std'],})
```

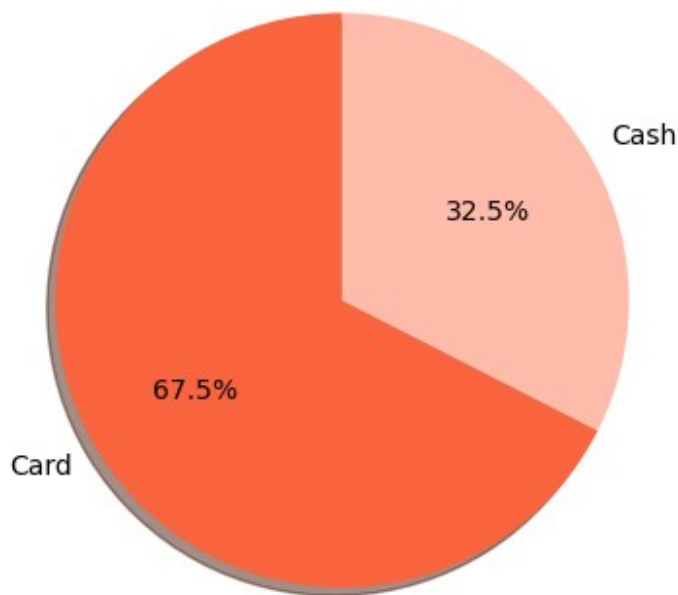


	fare_amount		trip_distance	
	mean	std	mean	std
payment_type				
Card	13.701903	6.506356	3.230729	2.320528
Cash	12.250209	6.246104	2.803716	2.231871

Now, in order to examine the passenger's preference regarding their choice of payment method, we will assess the proportion of the two payment types. To provide a visual representation, we have opted to utilize a pie chart. This graphical depiction will offer a clear and intuitive understanding of the distribution between the two payment methods chosen by passengers.

```
plt.title('Preference of Payment Type')
plt.pie(df['payment_type'].value_counts(normalize = True), labels =
df['payment_type'].value_counts().index,
startangle = 90,shadow = True, autopct = '%1.1f%%', colors =
['#FA643F', '#FFBCAB'])
plt.show()
```

Preference of Payment Type



Subsequently, we aim to conduct an analysis of the payment types in relation to the passenger count. Our objective is to investigate if there are any changes in preference contingent upon the number of passengers traveling in the cab.

To facilitate this examination, we have employed a visualization technique known as a stacked bar plot. This method is particularly advantageous for comparing the percentage distribution of each passenger count based on the payment method selected. Through this graphical representation, we can gain insights into potential variations in payment preferences across different passenger counts.

```
# calculating the total passenger count distribution based on the  
different payment type
```

```
passenger_count = df.groupby(['payment_type', 'passenger_count'])  
[['passenger_count']].count()  
passenger_count
```

		passenger_count
payment_type	passenger_count	
Card	1	953075
	2	336560
	3	124662
	4	64752
	5	126283
Cash	1	476475
	2	159243
	3	55491

4	33208
5	48469

```
passenger_count.columns
```

```
Index(['passenger_count'], dtype='object')
```

```
# renaming the passenger_count to count to reset the index
```

```
passenger_count.rename(columns = {'passenger_count': 'count'}, inplace = True)
```

```
passenger_count
```

payment_type	passenger_count	count
Card	1	953075
	2	336560
	3	124662
	4	64752
	5	126283
Cash	1	476475
	2	159243
	3	55491
	4	33208
	5	48469

```
passenger_count.reset_index(inplace = True)
```

```
passenger_count
```

	payment_type	passenger_count	count
0	Card	1	953075
1	Card	2	336560
2	Card	3	124662
3	Card	4	64752
4	Card	5	126283
5	Cash	1	476475
6	Cash	2	159243
7	Cash	3	55491
8	Cash	4	33208
9	Cash	5	48469

```
# calculating the percentage of the each passenger count
```

```
passenger_count['perc'] =
```

```
(passenger_count['count']/passenger_count['count'].sum())*100
```

```
passenger_count
```

	payment_type	passenger_count	count	perc
0	Card	1	953075	40.075174
1	Card	2	336560	14.151772
2	Card	3	124662	5.241824
3	Card	4	64752	2.722711

4	Card	5	126283	5.309984
5	Cash	1	476475	20.034959
6	Cash	2	159243	6.695896
7	Cash	3	55491	2.333302
8	Cash	4	33208	1.396340
9	Cash	5	48469	2.038039

creating a new empty dataframe to store the distribution of each payment type (useful for the visualization)

```
dff = pd.DataFrame(columns = ['payment_type',1,2,3,4,5])
dff
```

Empty DataFrame

Columns: [payment_type, 1, 2, 3, 4, 5]

Index: []

```
dff['payment_type'] = ['Card','Cash']
dff
```

	payment_type	1	2	3	4	5
0	Card	NaN	NaN	NaN	NaN	NaN
1	Cash	NaN	NaN	NaN	NaN	NaN

```
dff.iloc[0,1:] = passenger_count.iloc[:5,-1]
```

```
dff.iloc[1,1:] = passenger_count.iloc[5:-1]
```

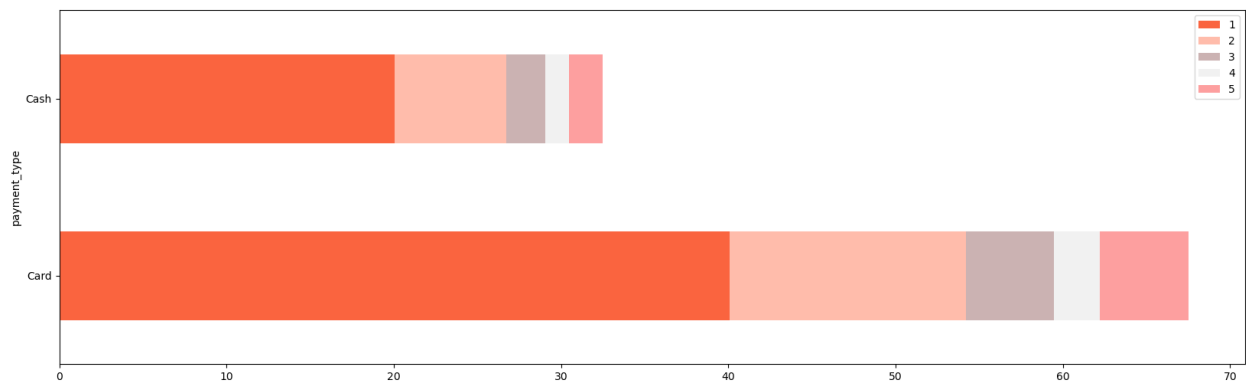
```
dff
```

	payment_type	1	2	3	4	5
0	Card	40.075174	14.151772	5.241824	2.722711	5.309984
1	Cash	20.034959	6.695896	2.333302	1.39634	2.038039

```
fig, ax = plt.subplots(figsize=(20, 6))
```

```
dff.plot(x='payment_type', kind='barh', stacked=True, title='',
ax=ax, color = ['#FA643F', '#FFBCAB', '#CBB2B2', '#F1F1F1', '#FD9F9F']
)
```

```
plt.show()
```



```

# fig, ax = plt.subplots(figsize=(20, 6))
# df.plot(x='payment_type', kind='barh', stacked=True, title=' ',
# ax=ax, color = ['#FA643F', '#FFBCAB', '#CBB2B2', '#F1F1F1', '#FD9F9F']
# )

# # Add percentage text
# for p in ax.patches:
#     width = p.get_width()
#     height = p.get_height()
#     x, y = p.get_xy()
#     ax.text(x + width / 2,
#             y + height / 2,
#             '{:.0f}%'.format(width),
#             horizontalalignment='center',
#             verticalalignment='center')

```

Hypothesis Testing

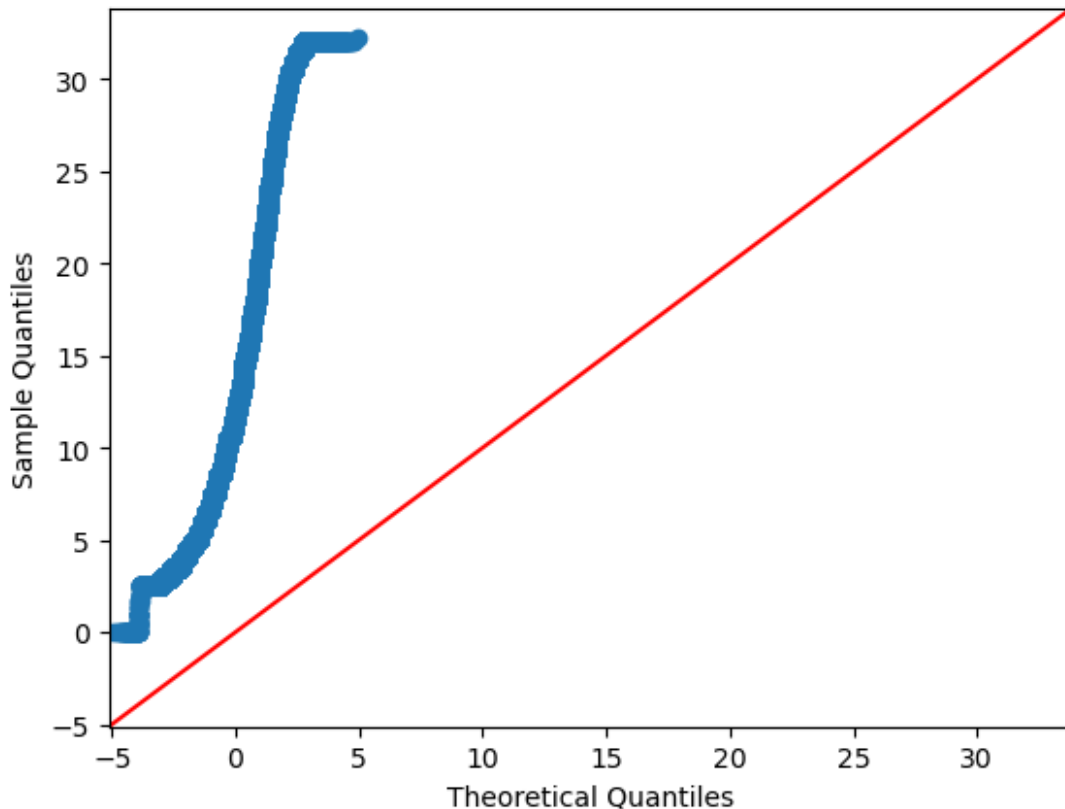
In order to select the most suitable test for our scenario, our initial step involves evaluating whether the distribution of fare amounts adheres to a normal distribution. While the histogram depicted above suggests otherwise, we will further confirm this by generating a QQ plot.

Quantile-quantile (QQ) plots can be used to assess whether the fare amount distributions for each payment type are approximately normally distributed. If the data points closely align with the diagonal line in the plot, it suggests that the data follows a normal distribution.

```

# create Q-Q plot with 45-degree line added to plot
fig = sm.qqplot(df['fare_amount'], line='45')
plt.show()

```



The data values clearly do not follow the red 45-degree line, which is an indication that they do not follow a normal distribution. So, z distribution will not be good for this. That's why we will use T test.

Given that the T-test can be applied to both small and large samples and does not require the population standard deviation, it is a more universally applicable approach for hypothesis testing in many practical research scenarios, including analyses of taxi trip data.

In the analysis of NYC Yellow Taxi Trip Records, where you're likely dealing with an unknown population standard deviation and potentially large datasets, the T-test offers a more appropriate and flexible method for comparing means between two groups (e.g., fare amounts by payment type). It provides a reliable way to infer about the population, accommodating the uncertainty that comes with estimating population parameters from sample data.

Null hypothesis: There is no difference in average fare between customers who use credit cards and customers who use cash.

Alternative hypothesis: There is a difference in average fare between customers who use credit cards and customers who use cash

```
credit_card = df[df['payment_type'] == 'Card']['fare_amount']  
credit_card
```

```
0      6.0  
1      7.0
```

```
2          6.0
3          5.5
9          8.0
```

```
...
6339550    10.5
6339555    10.0
6339561    17.5
6339563    13.0
6339564    12.5
```

```
Name: fare_amount, Length: 1605332, dtype: float64
```

```
# sample 2
```

```
cash = df[df['payment_type'] == 'Cash']['fare_amount']
cash
```

```
5          2.5
11         9.5
15         9.0
17        11.5
18        24.5
```

```
...
6339497    30.0
6339503    27.5
6339517    10.5
6339527    12.5
6339529    26.0
```

```
Name: fare_amount, Length: 772886, dtype: float64
```

```
# sample 1
```

```
credit_card = df[df['payment_type'] == 'Card']['fare_amount']
```

```
# sample 2
```

```
cash = df[df['payment_type'] == 'Cash']['fare_amount']
```

```
# performing t test on both the different sample
```

```
t_stat, p_value = stats.ttest_ind(a=credit_card, b=cash,
equal_var=False)
```

```
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

```
T-statistic: 165.59915491544467, P-value: 0.0
```

```
# comparing the p value with the significance of 5% or 0.05
```

```
if p_value < 0.05:
    print("\nReject the null hypothesis")
else:
    print("\nAccept the null hypothesis")
```

```
Reject the null hypothesis
```


Since the p-value is significantly smaller than the significance level of 5%, we will reject the null hypothesis.

You conclude that there is a statistically significant difference in the average fare amount between customers who use credit cards and customers who use cash.

The key business insight is that encouraging customers to pay with credit cards can generate more revenue for taxi cab drivers.

```
cleaned_path = r"C:\Users\Admin\Documents\Deepak Documents\PROJECTS\Hypothesis Project\cleaned_yellow_tripdata_2020-01.csv"
df.to_csv(cleaned_path, index=False)
```

Phase 2: Operational Intelligence (EDA & SQL)

Once the data is clean, answer these business questions. The Client asks:

1. The "Golden Hours": "When is our fleet busiest? I want a heatmap or bar chart showing the average number of trips by Hour of Day. Do we need more drivers at 5 AM or 5 PM?"
2. The Solo vs. Group Traveler: "Do people mostly travel alone or in groups? What is the most common passenger_count? Does the tip_amount increase if there are more passengers?"
3. The Payment Split: "What percentage of our customers pay by Credit Card vs. Cash (payment_type)? Is there a difference in the average tip_amount between card users and cash users?" (Hint: This is a classic behavioural insight.)
4. The Airport Run: "We suspect trips with RatecodeID = 2 (JFK Airport) have a higher average fare than standard city trips (RatecodeID = 1). Can you prove this with data?"

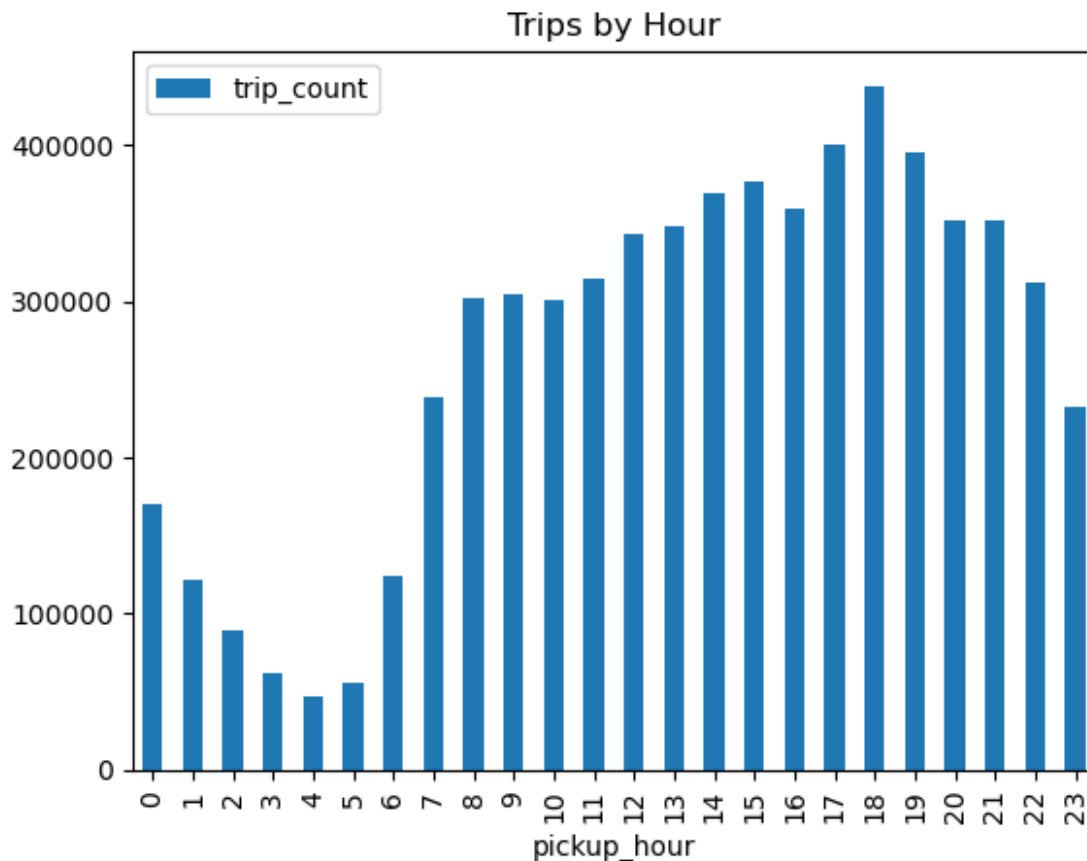
1 Golden Hours — Trips by Hour

```
df['pickup_hour'] = pd.to_datetime(df['tpep_pickup_datetime']).dt.hour

hourly_trips =
df.groupby('pickup_hour').size().reset_index(name='trip_count')

hourly_trips.plot(kind='bar', x='pickup_hour', y='trip_count',
title="Trips by Hour")

<Axes: title={'center': 'Trips by Hour'}, xlabel='pickup_hour'>
```



Solo vs Group Travelers — Passenger Behavior

```
passenger_dist = df['passenger_count'].value_counts()
```

```
avg_tip_by_passengers = df.groupby('passenger_count')  
['tip_amount'].mean()
```

```
passenger_dist, avg_tip_by_passengers
```

```
(passenger_count  
1.0    4547226  
2.0     946423  
3.0     250234  
5.0     225693  
6.0     132154  
4.0     123470  
0.0     114302  
7.0         29  
9.0         19  
8.0         17  
Name: count, dtype: int64,  
passenger_count
```

```

0.0    2.117647
1.0    2.214389
2.0    2.233318
3.0    2.161958
4.0    2.070792
5.0    2.219931
6.0    2.233537
7.0    6.552759
8.0    4.960000
9.0    7.793158
Name: tip_amount, dtype: float64)

```

Payment Split — Card vs Cash + Tip Difference

```

payment_split = df['payment_type'].value_counts(normalize=True) * 100
avg_tip_payment = df.groupby('payment_type')['tip_amount'].mean()
payment_split, avg_tip_payment

(payment_type
1.0    74.057061
2.0    25.141055
3.0     0.516912
4.0     0.284956
5.0     0.000016
Name: proportion, dtype: float64,
payment_type
1.0     2.985673
2.0     0.000341
3.0    -0.002215
4.0     0.024151
5.0     0.000000
Name: tip_amount, dtype: float64)

```

Airport Run — JFK vs City Fare Comparison

```

avg_fare_by_rate = df.groupby('RatecodeID')['fare_amount'].mean()
avg_fare_by_rate

RatecodeID
1.0    11.065083
2.0    51.078606
3.0    62.819188
4.0    73.580214

```

```
5.0      58.731113
6.0      30.905435
99.0     21.230000
Name: fare_amount, dtype: float64
```

Phase 3: The Strategic "So What?" (Storytelling)

Don't just give me a table. Give me a recommendation. The Client asks:

1. Congestion Impact: "We charge a congestion_surcharge. Does this surcharge discourage people? Compare the trip volume for trips with surcharge vs. without. What is your recommendation? Should we lobby to remove it?"
2. The "Whale" Locations: "Which Pickup Location (PULocationID) generates the highest total revenue? Identify the Top 3 locations and recommend where we should station our empty taxis."

1. Congestion Surcharge Impact

```
df['has_surcharge'] = df['congestion_surcharge'] > 0
trip_volume_surcharge = df['has_surcharge'].value_counts()
trip_volume_surcharge
has_surcharge
True      5845723
False     559285
Name: count, dtype: int64
```

2. Whale Locations — Top Revenue Pickup Zones

```
top_locations = df.groupby('PULocationID')
['total_amount'].sum().sort_values(ascending=False).head(3)
top_locations
PULocationID
132      11342435.83
138       5433833.23
161       4921563.36
Name: total_amount, dtype: float64
```

Phase 4: The New "Must-Haves" (Advanced Data Engineering)

The Client asks:

1. The "Time Travel" Bug (Duration Calculation): o "We need a trip_duration column (in minutes). But be careful—I ran a quick check and found trips with negative duration (e.g., Dropoff was before Pickup). Find them, count them, and kill them. They are corrupting our averages."
2. The "Rocket Car" Anomaly (Speed Analysis): o "Calculate the average speed (speed_mph) for every trip. o Logic Check: Speed = Distance / (Duration_in_Hours). o The Problem: We suspect some drivers are spoofing GPS. If a trip shows a speed > 100 MPH or < 0 MPH, flag it as 'Data Error'. How many rows do we lose?"

1. Trip Duration + Remove Negative Duration

```
df['pickup_time'] = pd.to_datetime(df['tpep_pickup_datetime'])
df['dropoff_time'] = pd.to_datetime(df['tpep_dropoff_datetime'])

df['trip_duration_min'] = (df['dropoff_time'] -
df['pickup_time']).dt.total_seconds() / 60

negative_trips = df[df['trip_duration_min'] < 0]

negative_count = len(negative_trips)

df = df[df['trip_duration_min'] >= 0]

negative_count

7
```

2. Speed Calculation + GPS Spoof Detection

```
df['duration_hours'] = df['trip_duration_min'] / 60

df['speed_mph'] = df['trip_distance'] / df['duration_hours']

df['speed_flag'] = df['speed_mph'].apply(
    lambda x: 'Data Error' if x > 100 or x < 0 else 'Valid'
)

error_speed_rows = df[df['speed_flag'] == 'Data Error']

error_speed_rows.shape[0]
```

Phase 5. The "Vendor Wars" (comparative Analysis)

The Client asks:

1. VeriFone (1) vs. Creative Mobile (2):
 - o "We use two providers for our in-car systems: Vendor 1 and Vendor 2.
 - o Vendor 2 handles ~70% of our rides. But who is actually better?
 - o The Challenge: Compare the Average Tip Amount for Vendor 1 vs. Vendor
2. Does the interface of one machine encourage better tipping? This is a huge potential revenue find for our drivers."

```
vendor_tip = df.groupby('VendorID')['tip_amount'].mean()
```

```
vendor_tip
```

```
VendorID
```

```
1.0    2.122029
```

```
2.0    2.255260
```

```
Name: tip_amount, dtype: float64
```

Phase 6. The "Mystery" Codes (Data Detective Work)

The Client asks:

1. The "RateCode 99" Mystery:
 - o "Standard trips are RateCode 1. JFK Airport is RateCode 2.
 - o But my tech team says we have 64 trips with RatecodeID = 99.
 - o What are these? Are they high value? Low value? Scams? Isolate these 64 rows and tell me what they have in common."

```
rate99 = df[df['RatecodeID'] == 99]
rate99.describe()
```

	VendorID	passenger_count	trip_distance	RatecodeID
PULocationID \				
count	350.000000	350.000000	350.000000	350.0
350.000000				
mean	1.734286	0.925714	1.226571	99.0
182.394286				
min	1.000000	0.000000	0.000000	99.0
7.000000				
25%	1.000000	0.000000	0.000000	99.0
132.000000				

50%	2.000000	1.000000	0.000000	99.0
190.500000				
75%	2.000000	1.000000	1.315000	99.0
239.000000				
max	2.000000	5.000000	30.060000	99.0
265.000000				
std	0.442345	0.734083	2.942607	0.0
70.983908				

	DOLocationID	payment_type	fare_amount	extra	mta_tax
...	\				
count	350.000000	350.000000	350.000000	350.000000	350.000000
...					
mean	186.720000	1.805714	21.230000	0.460571	0.430857
...					
min	7.000000	1.000000	-2.500000	-0.800000	-0.500000
...					
25%	137.000000	2.000000	4.000000	0.000000	0.500000
...					
50%	208.000000	2.000000	9.500000	0.000000	0.500000
...					
75%	264.000000	2.000000	24.875000	0.800000	0.500000
...					
max	265.000000	3.000000	400.000000	5.300000	3.300000
...					
std	74.375732	0.498676	35.858915	0.834080	0.460057
...					

	tolls_amount	improvement_surcharge	total_amount	\
count	350.000000	350.000000	350.000000	
mean	0.122400	0.172286	24.037657	
min	0.000000	0.000000	-3.800000	
25%	0.000000	0.000000	7.300000	
50%	0.000000	0.300000	12.700000	
75%	0.000000	0.300000	26.637500	
max	6.120000	0.300000	400.000000	
std	0.858027	0.148548	35.913862	

	congestion_surcharge	pickup_hour	
pickup_time \			
count	350.000000	350.000000	
350			
mean	1.292857	13.237143	2020-01-16
10:27:35.937142784			
min	0.000000	0.000000	2020-01-01
00:24:18			
25%	0.000000	10.000000	2020-01-08
15:20:33.249999872			
50%	2.500000	14.000000	2020-01-16
13:09:50			

75%	2.500000	17.000000	2020-01-24
06:53:38.750000128			
max	2.500000	23.000000	2020-01-31
21:04:18			
std	1.251054	5.516663	
NaN			

		dropoff_time	trip_duration_min	duration_hours
\				
count		350	350.000000	350.000000
mean	2020-01-16	11:17:56.871428608	50.348905	0.839148
min	2020-01-01	00:52:47	0.000000	0.000000
25%	2020-01-08	16:12:54	0.087500	0.001458
50%	2020-01-16	13:53:30.500000	5.591667	0.093194
75%	2020-01-24	07:00:51.249999872	12.162500	0.202708
max	2020-01-31	21:06:09	4168.000000	69.466667
std		NaN	338.567574	5.642793

	speed_mph
count	265.000000
mean	7.799842
min	0.000000
25%	0.000000
50%	6.171429
75%	11.250000
max	44.016194
std	9.231053

[8 rows x 21 columns]

```
rate99[['fare_amount', 'trip_distance', 'tip_amount', 'PULocationID']].head()
```

	fare_amount	trip_distance	tip_amount	PULocationID
1968	17.0	2.15	0.0	246
13339	9.5	1.90	0.0	164
22212	12.0	2.56	0.0	80
28534	2.5	0.00	0.0	144
31133	5.0	0.66	0.0	68

Phase 7. The "Route Master" (Geospatial Logic)

The Client asks:

1. The "Commuter Loop": o "Find the Top 5 most popular pairs of Pickup (PULocationID) and Dropoff (DOLocationID) locations.

o Hint: Create a new column route = PULocationID -> DOLocationID.

o The Question: Are the top routes long cross-city trips, or just short hops (e.g., 237 -> 236)? If they are short hops, we should deploy electric scooters there instead of taxis."

```
# create a route column
df['route'] = df['PULocationID'].astype(str) + " -> " +
df['DOLocationID'].astype(str)
```

Top 5 Popular Routes

```
top_routes = df['route'].value_counts().head(5)
top_routes
```

```
route
237 -> 236    45539
236 -> 236    38775
236 -> 237    38264
237 -> 237    33909
264 -> 264    27928
Name: count, dtype: int64
```

```
route_distance = df.groupby('route')
['trip_distance'].mean().sort_values(ascending=False).head(5)
route_distance
```

```
route
76 -> 231    7257.239655
265 -> 39     61.420000
220 -> 68     52.260000
9 -> 44       48.653333
181 -> 265    47.173333
Name: trip_distance, dtype: float64
```

SQL Equivalent

Golden Hours SQL

```
# SELECT HOUR(tpep_pickup_datetime) AS hour, COUNT(*)  
# FROM taxi_trips  
# GROUP BY hour  
# ORDER BY hour
```

Payment Tip Comparison SQL

```
# SELECT payment_type, AVG(tip_amount)  
# FROM taxi_trips  
# GROUP BY payment_type;
```

Top Revenue Locations SQL

```
# SELECT PULocationID, SUM(total_amount)  
# FROM taxi_trips  
# GROUP BY PULocationID  
# ORDER BY SUM(total_amount) DESC  
# LIMIT 3;
```